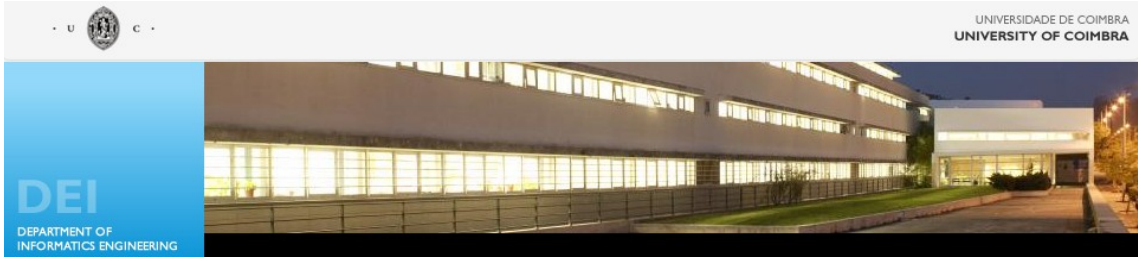

Desempenho de Serviços e Redes de Comunicação

(Communication Networks and Services Performance) [2025-2026]

Exercise 01 – Modelling your network

Introduction

This exercise aims at learning the necessary steps to tailor the network and workload modelling to specific needs. During this assignment, students will get familiarised with the definition of the network topology, workloads, and incoming requests.

Objectives

Students successfully concluding this work should be able to:

- Understand the configuration of a network topology using YAFS
- Understand the configuration of the workload and requests using YAFS
- Practice with tailored scenarios is YAFS

Submission

Groups of 2 students shall submit the work in *Inforestudante* as a .zip, containing the following files:

- 2 page A4 report detailing the process used on each exercise
- All the plots obtained
- All the JSON and Python files changed/created

Submission deadline: **March 16th, 2026**.

Background

JSON files

The JavaScript Object Notation (JSON) is known as the data interchange format standard [1]. It can be used wherever the exchange of data occurs. It is a textual representation defined by a small set of governing rules in which data is structured, The JSON specification states that data can be structured in either of the two following compositions:

- A collection of name/value pairs
- An ordered lists of values

Examples of valid representations of a collection of Key/Value pairs, per JSON Grammar, are depicted in Figure 1.

```
//Empty collection set
{};
//Single string/value pair
{"abc":"123"};
//Multiple string/value pairs
{"captainsLog":"startDate 9522.6","message":"I've never trusted Klingons, and I never
will."};
```

Figure 1. Examples of valid representations of a collection of Key/Value pairs, per JSON Grammar

Figure 2 offers examples of representations for ordered lists.

```
//Empty ordered list
[];
//Ordered list of single value
["abc"];
//Ordered list of multiple values
["0",1,2,3,4,100];
```

Figure 2. Examples of valid representations of an ordered list, per JSON Grammar

It is possible to use editors, validators, and formatting tools for JSON files. Look for some online options![1,2,3]

[1] https://jsonlint.com/

[2] https://jsonformatter.curiousconcept.com/

[3] https://jsonformatter.org/

Activities

1. Updating the topology

Goal: Learn how to tailor your network topology, manually or dynamically using NetworkX [1].

Go to the `tutorial_scenarios/01_basicExample` folder. Take a look at the file `main.py`. Identify where is the topology defined.

What topology type is being used?

NetworkX is being used to create the topology. Use **two** of the following generators[4]:

- balanced_tree
- complete_graph
- random_regular_graph
- barabasi_albert_graph
- tadpole_graph
- star_graph

Describe the parameters and attach the generated plots.

2. Customising the topology

Goal: Learn how to configure the resources capabilities of the nodes and links in the topology.

On the same file, `main.py`, identify where is the topology customised (i.e., links: BW, PR; nodes: IPT, RAM, COST). Remember that the nodes are called *entities.* Alter these values and run the simulation.

Describe the parameters and difference in results.

3. Optimising your workflow with JSON files

Goal: Learn how to use JSON files to define your topology.

Use file `networkDefinition.json` provided for this exercise. Analyse the JSON file. Make the necessary changes in your `main.py` file so the topology is created from the JSON file instead of using NetworkX.

What is the topology created? What is its structure? What is its size?

Submit the generated plot and Python file.

[4] https://networkx.org/documentation/stable/reference/generators.html#graph-generators

4. Creating your workload

Goal: Learn how to define the workload for the simulation.

The workload refers to the applications deployed in your topology. The applications are composed by microservices called *modules*. Search for "`APPLICATION or SERVICES`" in your `main.py` file. The applications are defined in a json file, `appDefinition.json`. Open this file and analyse it. Try the file `appDefinition2.json` provided with this exercise.

Explain the differences.

Create a JSON file with three applications:

- App1: one module
- App2: three modules
- App3: five modules

Submit the JSON file.

5. Dynamic workload

Goal: Learn how to generate dynamic workloads.

Create a Python script that generates a new JSON file, `dynamicAppDefinition.json`, that allows the parametrised and randomised generation of workloads.

Submit the Python script and generated JSON file.

6. Generating requests: User population

Goal: Learn how to generate requests, i.e., initial messages generated from an app.

We will now define the initial messages (i.e., requests) for each app deployed in our topology. The workload refers to the applications deployed in your topology. The applications are composed by microservices called *modules*. Search for "`DEPLOY USERS`" in your `main.py` file. The users are defined in a json file, `usersDefinition.json`. Open this file and analyse it. Try the file `usersDefinition2.json` provided with this exercise.

Explain the differences.

7. Dynamic population

Goal: Learn how to generate dynamic requests.

Create a Python script that generates a new JSON file, `dynamicUsersDefinition.json`, that allows the parametrised and random generation of requests.

Submit the Python script and generated JSON file.

References

[1] A. Harberg, D. Schult and P. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proceedings of the 7th Python in Science Conference (SciPy2008)*, Pasadena, CA, 2008.

[2] "YAFS Examples," [Online]. Available: https://yafs.readthedocs.io/en/latest/examples/index.html.

[3] I. Lera and C. Guerrero, "YAFS Tutorial," [Online]. Available: https://yafs.readthedocs.io/en/latest/introduction/index.html.

[4] I. Lera and C. Guerrero, "YAFS User Guide," [Online]. Available: https://www.slideshare.net/wisaaco/yet-another-fog-simulator-yafs-user-guide.

[5] B. Smith, Beginning JSON, Apress, 2015.